

AI Math Assistant

A LangChain + OpenAI agent with add, subtract, multiply and divide tools, run as a plain Python file

1. What this is

math_assistant.py replaces the notebook. It defines four math *tools*, gives them to an OpenAI model through LangChain's `create_agent`, and lets you chat with it in the terminal. The model decides which tool to call, in which order, and then answers in plain English.

- **add(a, b)** returns $a + b$
- **subtract(a, b)** returns $a - b$
- **multiply(a, b)** returns $a * b$
- **divide(a, b)** returns a / b (returns an error message instead of crashing when b is 0)

2. Where the files are

- Code: `D:\development\ibm_rag_plus_agentic_ai\ai-math-assistant-project13\math_assistant.py`
- API key: `.env` in the same folder (already ignored by git)
- This guide: `AI_Math_Assistant_Instructions.pdf` in the project folder (a copy is also on your Desktop)

3. One-time setup

The project already has everything installed. If you set it up on another computer, run these in PowerShell:

```
uv init ai-math-assistant --python 3.12
cd ai-math-assistant
uv add langchain langchain-openai python-dotenv
```

Then create a file named `.env` next to `math_assistant.py` containing:

```
OPENAI_API_KEY=sk-...your key...
```

4. Run it

Open a terminal in the project folder (in VS Code: **Terminal** → **New Terminal**) and run:

```
uv run python math_assistant.py
```

Example session:

```
AI Math Assistant (type 'quit' to exit)

You: what is 100 minus 37?
Assistant: 100 minus 37 is 63.

You: add 25 and 15, then multiply by 2, then divide by 4
Assistant:  $(25 + 15) \times 2 / 4 = 20$ .

You: quit
Goodbye!
```

4b. Tracing

Local trace: add `--trace` to see every tool call and its result:

```
uv run python math_assistant.py --trace

You: add 25 and 15, then multiply by 2, then divide by 4
[trace] call    add({'a': 25, 'b': 15})
[trace] result add -> 40.0
[trace] call    multiply({'a': 40, 'b': 2})
[trace] result multiply -> 80.0
[trace] call    divide({'a': 80, 'b': 4})
[trace] result divide -> 20.0
Assistant: 20
```

LangSmith trace (web dashboard with every prompt, tool call, token count and timing): create a free account at smith.langchain.com, create an API key, and add these lines to `.env`. No code change is needed; LangChain sends traces automatically. Remove the lines (or set `LANGSMITH_TRACING=false`) to turn it off.

```
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=lsv2...your key...
LANGSMITH_PROJECT=ai-math-assistant
```

5. Stopping it

- Type **quit**, **exit** or **q** and press Enter.
- Or press **Ctrl+C** at any time, including while it is still thinking. Unlike a notebook, there is no kernel to interrupt; Ctrl+C stops the program immediately.
- The agent also stops itself after 20 steps (`recursion_limit`), so it can never loop forever.

6. How it works

- **@tool** turns each Python function into a LangChain tool. The type hints (`a: float, b: float`) and the docstring tell the model what the tool does and what to pass.
- **ChatOpenAI** is the language model. Change `OPENAI_MODEL` at the top of the file to use another model.
- **create_agent** connects the model and the tools. The system prompt tells it to always use the tools for arithmetic.
- **ask()** sends one question to the agent and returns the last message, which is the final answer.
- **main()** is the chat loop: read a question, ask the agent, print the answer.

7. Adding your own tool

Write a function, decorate it with `@tool`, give it a docstring, and add it to the `TOOLS` list. For example, a power tool:

```
@tool
def power(base: float, exponent: float) -> float:
    """Raise base to the power of exponent."""
    return base ** exponent

TOOLS = [add, subtract, multiply, divide, power]
```

8. Troubleshooting

- **OPENAI_API_KEY not found:** the `.env` file is missing or not in the folder you ran the command from.
- **ModuleNotFoundError:** run with `uv run python ...`, not plain `python`, so the project's `.venv` is used.
- **Model not found / 401 error:** check the key and the `OPENAI_MODEL` name.

9. Full code: `math_assistant.py`

```
"""AI Math Assistant: a LangChain agent with add, subtract, multiply and divide tools.
```

```
Run from the project folder:
```

```
    uv run python math_assistant.py          # normal
    uv run python math_assistant.py --trace  # also print each tool call and result
"""
```

```
import os
import sys
```

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
```

```
OPENAI_MODEL = "gpt-5.4-nano" # any tool-calling OpenAI chat model works, e.g. "gpt-4o-mini"
```

```
# -----
# Math tools
# -----
```

```
@tool
def add(a: float, b: float) -> float:
    """Add two numbers and return a + b."""
    return a + b
```

```
@tool
def subtract(a: float, b: float) -> float:
    """Subtract the second number from the first and return a - b."""
    return a - b
```

```
@tool
def multiply(a: float, b: float) -> float:
    """Multiply two numbers and return a * b."""
    return a * b
```

```
@tool
def divide(a: float, b: float) -> float | str:
    """Divide the first number by the second and return a / b.
    Returns an error message if b is zero."""
    if b == 0:
```

```

        return "Error: cannot divide by zero."
    return a / b

TOOLS = [add, subtract, multiply, divide]

SYSTEM_PROMPT = (
    "You are a helpful math assistant. Always use the provided tools for arithmetic "
    "instead of calculating in your head. For multi-step problems, call the tools one "
    "step at a time, then give the final answer in one short plain-text sentence (no LaTeX)."
```

)

Agent

def build_agent():
 load_dotenv() # reads OPENAI_API_KEY from the .env file
 if not os.getenv("OPENAI_API_KEY"):
 raise RuntimeError("OPENAI_API_KEY not found. Add it to the .env file in the project folder.")

 llm = ChatOpenAI(model=OPENAI_MODEL, temperature=0)
 return create_agent(model=llm, tools=TOOLS, system_prompt=SYSTEM_PROMPT)

def ask(agent, question: str, trace: bool = False) -> str:
 # recursion_limit stops the agent if it ever gets stuck in a loop of tool calls
 answer = ""
 for update in agent.stream(
 {"messages": [{"role": "user", "content": question}]},
 config={"recursion_limit": 20},
 stream_mode="updates",
):
 for step in update.values():
 for msg in (step or {}).get("messages", []):
 if msg.type == "ai":
 for call in msg.tool_calls:
 if trace:
 print(f" [trace] call {call['name']}({call['args']})")
 if not msg.tool_calls:
 answer = msg.content
 elif msg.type == "tool" and trace:
 print(f" [trace] result {msg.name} -> {msg.content}")
 return answer

def langsmith_enabled() -> bool:
 return os.getenv("LANGSMITH_TRACING", "").lower() == "true" and bool(os.getenv("LANGSMITH_API_KEY"))

def main():
 trace = "--trace" in sys.argv
 agent = build_agent()
 print("AI Math Assistant (type 'quit' to exit)")
 print("Local trace:", "on" if trace else "off (run with --trace to see tool calls)")
 if langsmith_enabled():
 print("LangSmith trace: on, project:", os.getenv("LANGSMITH_PROJECT", "default"))
 print("Example: add 25 and 15, then multiply by 2\n")

 while True:
 try:
 question = input("You: ").strip()
 except (KeyboardInterrupt, EOFError):
 break
 if question.lower() in {"quit", "exit", "q"}:
 break
 if not question:

```
        continue
    try:
        print("Assistant:", ask(agent, question, trace), "\n")
    except Exception as e:
        print("Error:", e, "\n")

print("Goodbye!")

if __name__ == "__main__":
    main()
```